

Colecciones y Genéricos

Sobreescribir hashCode(), equals() y toString()

Comprender **hashCode()** y **equals()** es esencial para trabajar con las colecciones de Java, especialmente cuando usamos **Maps** y cuando buscamos u ordenamos en general. Algunos métodos de la clase **Object**:

Method	Description
<code>boolean equals (Object obj)</code>	Decides whether two objects are meaningfully equivalent
<code>void finalize()</code>	Called by the garbage collector when the garbage collector sees that the object cannot be referenced (rarely used, and deprecated in Java 9)
<code>int hashCode()</code>	Returns a hashcode <code>int</code> value for an object so that the object can be used in Collection classes that use hashing, including <code>Hashtable</code> , <code>HashMap</code> , and <code>HashSet</code>
<code>final void notify()</code>	Wakes up a thread that is waiting for this object's lock
<code>final void notifyAll()</code>	Wakes up <i>all</i> threads that are waiting for this object's lock
<code>final void wait()</code>	Causes the current thread to wait until another thread calls <code>notify()</code> or <code>notifyAll()</code> on this object
<code>String toString()</code>	Returns a "text representation" of the object

El método **toString()**

El método **toString()**, si no es sobrescrito, suministra el nombre de la clase a la que pertenece el objeto, una **@**, y una representación hexadecimal sin signo del hashcode del objeto.

```
public class Elemento{
    public static void main(String[] args){
        Elemento el =new Elemento();
        System.out.println(el); // Invoca a toString()
    }
}
```

Es mejor sobrescribirlo para obtener una descripción más detallada y significativa de los objetos de nuestra clase.

```

public class BobTest {
    public static void main(String[ ] args){
        Bob f = new Bob("GoBobGo",19);
        System.out.println(f);
    }
}
class Bob {
    int shoeSize;
    String nickName;
    Bob(String nickname, int shoeSize){
        this.shoeSize = shoeSize;
        this.nickName = nickName;
    }
    public String toString(){
        return ("Me llamo Manolo, pero puedes llamarme " + nickName + ". Mi número de
        zapato es " + shoeSize);
    }
}

```

Sobrescribiendo equals()

Cuando quiera que necesitemos ordenar o buscar a través de los objetos de una colección, los métodos **equals()** y **hashCode()** son esenciales.

Cuando necesitamos saber si dos referencias son idénticas usamos **==**. Devolverá true si las dos referencias apuntan al mismo objeto porque simplemente mira a los bits en la variable, y si son idénticos o no. Cuando necesitamos saber si dos objetos son iguales (no sus referencias), usamos el método **equals()**. Para cada clase que escribimos, debemos decidir si tiene sentido considerar a dos instancias distintas como iguales.

El método **equals()** en la clase Object usa el operador **==** para las comparaciones, así que, a menos que sobrescribamos **equals()**, dos objetos serán considerados iguales sólo si las dos referencias usadas corresponden al mismo objeto. Para cada clase que se escriba, se debe decidir si tiene sentido considerar dos instancias distintas como iguales (es decir, que pasen el test **equals()**).

Implementando un método equals()

Podemos sobrescribir **equals()** en nuestra clase. Quedaría parecido a esto:

```

public class EqualsTest{

    public static void main(String[ ] args){
        Moof one = new Moof(8);
        Moof two = new Moof(8);
    }
}

```

```

    if(one.equals(two)){
        System.out.println("One y Two son iguales.");
    }
}
}

class Moof {
    private int moofValue;
    Moof(int val){
        moofValue = val;
    }

    public int getMoofValue( ){
        return moofValue;
    }

    public boolean equals(Object o){
        if(( o instanceof Moof) && (( Moof)o).getMoofValue( ) == this.moofValue )
        {
            return true;
        }
        else
        {
            return false;
        }
    }
}
}

```

En este caso hemos decidido que dos objetos de la clase son iguales si tiene el mismo valor para la variable `moofValue`, argumento del constructor. Fijémonos además que el argumento de **`equals( )`** es un **Object**, por lo que es necesario hacer un casting.

Hay que recordar que los métodos **`equals( )`**, **`hashCode( )`** y **`toString( )`** son todos **public**, por lo tanto hay que declararlos **public** al sobrescribirlos.

El contrato **`equals( )`**

Tomado de la documentación de Java, el contrato **`equals( )`**, para cualesquiera valores de referencia **`x`**, **`y`** y **`z`**, dice:

- **Es reflexivo:** Para cualquier valor de referencia **`x.equals(x)`** debe retornar true.
- **Es simétrico:** Para cualesquiera valores de referencia **`x.equals(y)`** si y sólo si **`y.equals(x)`**.
- **Es transitivo:** Para cualesquiera valores de referencia si **`x.equals(y)`** devuelve true y **`y.equals(z)`** devuelve true entonces **`x.equals(z)`** devuelve true.

- **Es consistente:** Para cualesquiera valores de referencia múltiples invocaciones de **x.equals(y)** devuelven consistentemente true o devuelven consistentemente false.
- Para cualquier valor de referencia x, **x.equals(null)** debe retornar false.

Si dos objetos son considerados iguales según el test **equals()**, entonces estos deben tener el mismo valor de **hashCode()** (Si se sobrescribe **equals()** es aconsejable sobrescribir **hashCode()**).

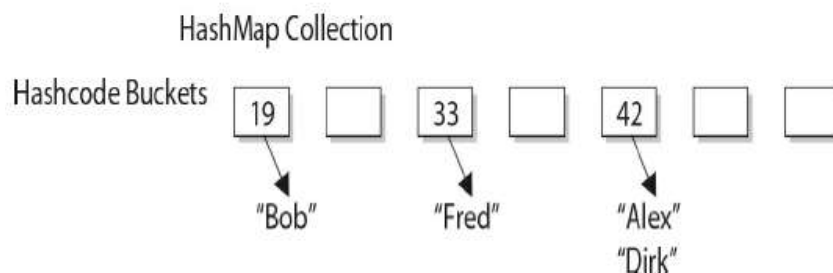
Sobreescribiendo hashCode()

Los hashcodes son usados típicamente para incrementar el rendimiento de grandes colecciones de datos. Aunque pensar que es una especie de ID única para un objeto, no es necesariamente única. Las colecciones tales como **HashMap** y **HashSet** (y en general los que contengan la palabra hash) usan el valor de hashCode de un objeto para determinar como este se debería almacenar en la colección, y también se usa el valor del hashCode para ayudar a localizar el objeto en la colección.

Comprendiendo los hashcodes

Imaginemos unos casilleros postales alineados en la pared, con el código numérico de su propietario escrito en un papel pegado en el frontal. El código de cada propietario se calcula tomando cada nombre y calculando una suma a partir de las letras del nombre (1 para A, 2 para B, 3 para C, etc,...). Mayúsculas y minúsculas tienen el mismo código, por simplicidad.

Key	HashCode Algorithm	HashCode
Alex	$A(1) + L(12) + E(5) + X(24)$	= 42
Bob	$B(2) + O(15) + B(2)$	= 19
Dirk	$D(4) + I(9) + R(18) + K(11)$	= 42
Fred	$F(6) + R(18) + E(5) + D(4)$	= 33



Sin embargo hay un problema, ya que los nombres May y Amy tendrán el mismo código numérico. Por lo tanto dentro de un mismo casillero, puede haber correo de más de una persona. Por tanto, para buscar el correo un usuario puede localizar su casillero por el código numérico, pero una vez localizado aún tendrá que ver que

correo va dirigido a él. El **hashCode()** sirve para hacer una primera búsqueda de un objeto situándolo en un conjunto de objetos similares. En una segunda fase se usa **equals()** para localizar el objeto concreto dentro del grupo.

No importa como de iguales sean los objetos si sus hashcodes no lo reflejan: Si son objetos satisfacen **equals()**, sus hashcodes deben también ser iguales.

El ejemplo es similar a la forma de usar hashcodes en las colecciones. Cuando ponemos un objeto en una colección que usa hashcodes, la colección usa el hashCode del objeto para decidir en qué casillero colocarlo. Después, cuando queremos recuperar el objeto tendremos que dar a la colección una referencia al objeto, que se compara con los objetos guardados en la colección usando su hashCode.

Implementando hashCode()

La implementación del hashCode() debería usar las mismas variables de instancia que se usen para sobrescribir equals().

```
class HasHash {
    public int x;
    HasHash(int xVal) { x = xVal; }
    public Boolean equals(Object o) {
        HasHash h = (HasHash) o;
        if ( h.x == this.x){
            return true;
        } else {
            return false;
        }
    }
    public int hashCode( ) { return (x*17); } // Legal, pero ineficiente.
}
```

Este método equals() dice que dos objetos son iguales si tienen el mismo valor que x, así que los objetos con el mismo valor de x deben retornar hashcodes idénticos.

El contrato hashCode()

- Cuando quiera que el **hashCode()** es invocado en el mismo objeto más de una vez durante la ejecución de una aplicación de Java, este debe devolver consistentemente el mismo entero, siempre que ninguna información usada en la comparación **equals()** sea modificada. No es necesario que el entero permanezca consistente de una ejecución de la aplicación a otra.

- Si dos objetos son iguales de acuerdo con el método **equals(Object)**, entonces al llamar al método hashCode en cada uno de los dos objetos debe de producir el mismo número entero como resultado.
- No es obligatorio que si dos objetos no son iguales según el método equals(), llamando al método hashCode en cada uno de los dos objetos deban producir números enteros distintos como resultado. Sin embargo, el programador debería darse cuenta de que producir distintos resultados enteros para objetos no iguales puede mejorar el rendimiento.

Condition	Required	Not Required (But Allowed)
<code>x.equals(y) == true</code>	<code>x.hashCode() == y.hashCode()</code>	
<code>x.hashCode() == y.hashCode()</code>		<code>x.equals(y) == true</code>
<code>x.equals(y) == false</code>		No hashCode() requirements
<code>x.hashCode() != y.hashCode()</code>	<code>x.equals(y) == false</code>	

Colecciones

Operaciones básicas con colecciones

- Añadir objetos a la colección.
- Eliminar objetos de la colección.
- Verificar si un objeto (o un grupo de objetos) está en la colección.
- Recuperar un objeto de la colección (sin eliminarlo de esta).
- Iterar a través de la colección, mirando cada elemento uno tras otro.

Interfaces y clases clave del Framework Collections

La API Collections comienza con un grupo de interfaces, cuyo núcleo son los siguientes:

Collection	Set	SortedSet
List	Map	SortedMap
Queue	NavigableSet	NavigableMap

Y un montón de clases de implementación. Las más usadas:

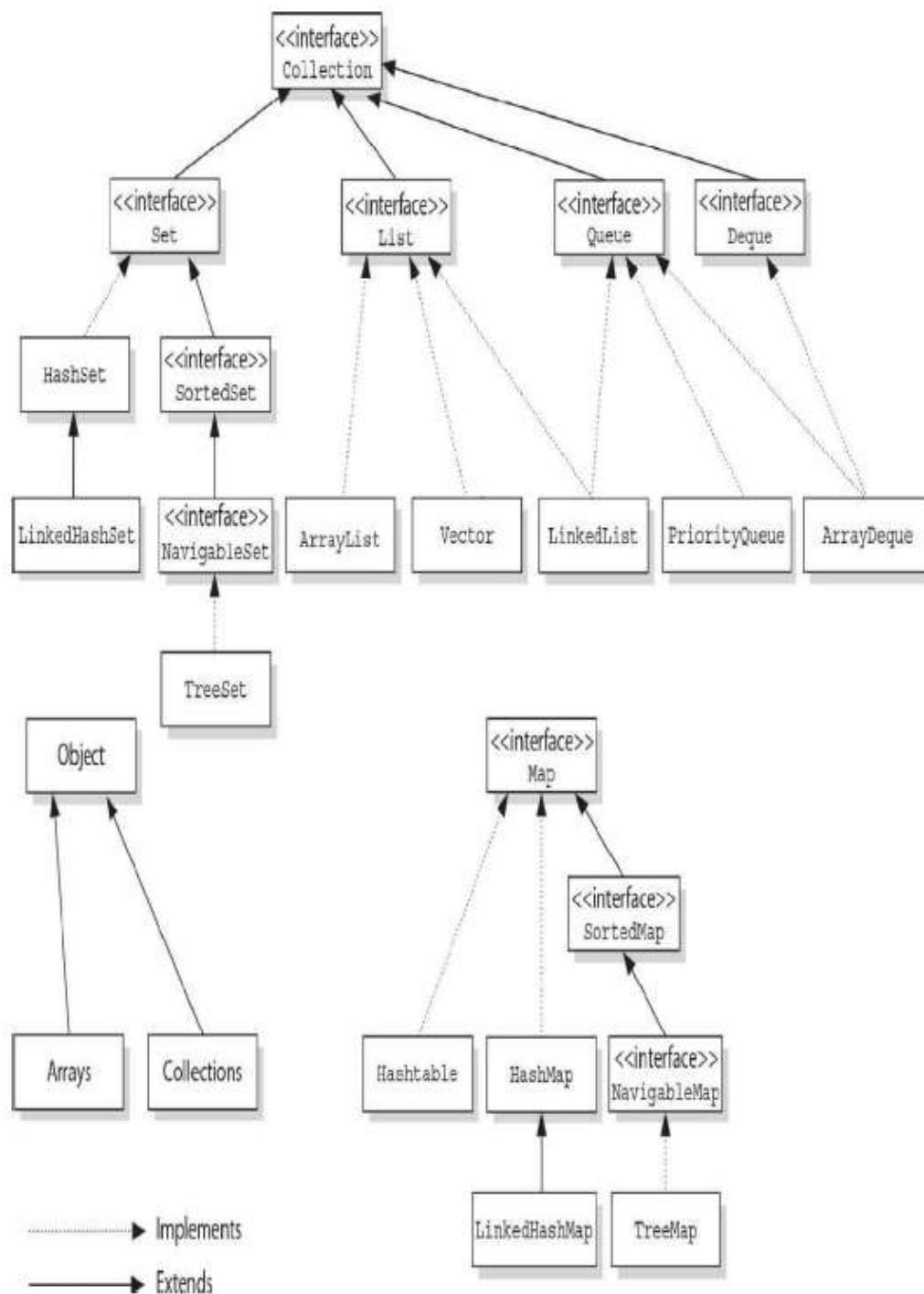
Maps	Sets	Lists	Queues	Utilities
HashMap	HashSet	ArrayList	PriorityQueue	Collections
Hashtable	LinkedHashSet	Vector	ArrayDeque	Arrays
TreeMap	TreeSet	LinkedList		
LinkedHashMap				

Oracle considera esenciales: **ArrayList**, **ArrayDeque**, **TreeMap**, **TreeSet** y **Arrays**.

Oracle considera importantes a: **Collections**, **HashMap**, **Hashtable** y **PriorityQueue**.

- **collection** (c minúscula) Representa cualquier estructura de datos en la cual los objetos son almacenados y se itera sobre ellos.
- **Collection** (C mayúscula) Es de hecho el interfaz de **java.util.Collection** del cuál extienden Set, List y Queue. No hay implementaciones directas de Collection.
- **Collections** (C mayúscula y acabado en s) Es la clase de **java.util.Collections** que mantiene un conjunto de métodos static de utilidad para usar con colecciones.

No hay que confundir **Collections** con **Collection**. Collections es una clase con métodos static de utilidad, mientras que Collection es un interface con declaraciones de los métodos más comunes a las colecciones, tales como **add()**, **contains()**, **remove()**, **size()** e **iterator()**.

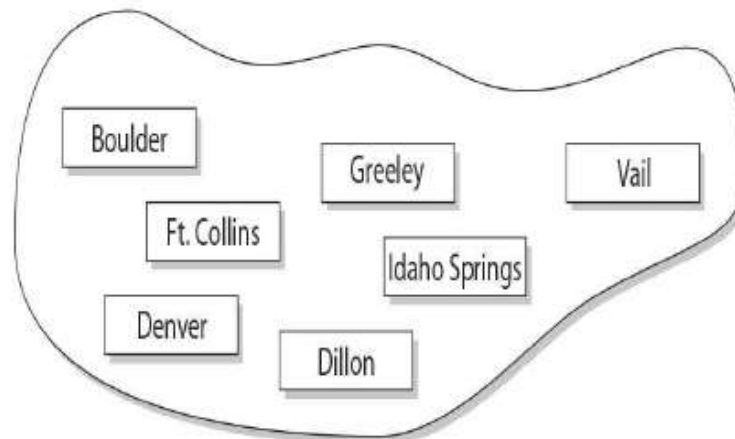


Las colecciones tienen cuatro sabores básicos:

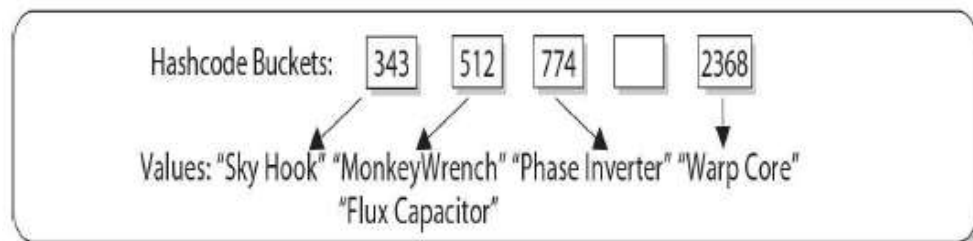
- **Lists** (Listas de cosas, con un índice). Implementan List.
- **Sets** (Conjuntos de cosas únicas). Implementan Set.
- **Maps** (Cosas con un identificador único). Implementan Map.
- **Queues** (Cosas dispuestas en orden que han de ser procesadas).

Index:	0	1	2	3	4	5
Value:	"Boulder"	"Ft. Collins"	"Greeley"	"Boulder"	"Denver"	"Boulder"

List: The salesman's itinerary (Duplicates allowed)



Set: The salesman's territory (No duplicates allowed)



HashMap: The salesman's products (Keys generated from product IDs)

Pero además hay subsabores dentro de los sabores de las colecciones: **SORTED, UNSORTED, ORDERED, UNORDERED**

Una clase de implementación puede ser unsorted y unordered, ordered pero unsorted, o ambos ordered y sorted. Pero nunca puede ser sorted pero unordered, por que la clasificación (sorting) es un tipo específico de ordenación (ordering).

Ordered Cuando una colección está ordenada (ordered) esto significa que podemos iterar a través de ella en un orden específico (no aleatorio). Un ArrayList guarda el orden establecido por el índice de posición de los elementos (como un array). LinkedHashSet mantiene el orden establecido por la inserción (el último elemento

insertado es el último elemento insertado en el `LinkedHashSet` (al contrario que `ArrayList`, en donde podemos insertar un elemento en una posición de índice específica). Algunas colecciones guardan un orden llamado **orden natural** de los elementos y esas colecciones no son sólo ordenadas, sino además clasificadas.

Sorted Una colección clasificada (sorted) significa que su orden en la colección está determinado por alguna regla o por algún conjunto de reglas, conocido como orden de clasificación. El orden de clasificación no tiene nada que ver con cuando se añadió el objeto a la colección, o cuando fue accedido por última vez o en qué posición fue añadido. La clasificación está basada en propiedades de los propios objetos. El orden obtenido de la clasificación es conocido normalmente como **orden natural**.

En la clase **String** el orden natural ya está establecido (orden alfabético) y también para las clases de envoltura numéricas, pero en el resto de las clases que nosotros definamos el orden natural no está establecido y deberemos establecerlo implementando el interface **Comparable** (que define como una instancia de una clase puede ser comparada con otra de esa misma clase). Por supuesto, en algunos casos, nuestra clase puede heredar su orden natural de una superclase en vez de definir uno propio.

Aparte de definir el orden natural para las instancias de una clase usando el interfaz **Comparable**, también es posible definir otros ordenes de clasificación usando otro interface, el interface **Comparator**.

Métodos de la interface Collection

- `public boolean add(Object o)`
- `public boolean remove(Object o)`
- `public boolean contains(Object o)`
- `public int size( )`
- `public boolean isEmpty( )`
- `public Iterator iterator( )`
- `public Object[ ] toArray( )`
- `public Object[ ] toArray(Object[ ] a)`

Interfaz List

Una lista se rige por un índice. Por ello tiene un conjunto de métodos relacionados con un índice que no tienen las demás interfaces. Las tres implementaciones de **List** están ordenadas por la posición del índice y son **ordenadas, pero no clasificadas**. Algunos métodos de la interface **List**:

- **add(Object o)**: añade un objeto al final de la lista.
- **add(int índice, Object o)**: añade un objeto a la lista en la posición indicada.
- **get(int índice)**: devuelve el objeto de la lista de la posición indicada.
- **remove(int índice)**: elimina el objeto de la lista pasado por parámetro.

- **clear()**: elimina todos los elementos de la lista.
- **indexOf(Object o)**: devuelve la posición de la primera vez que un elemento coincida con el objeto pasado por parámetro. Si el elemento no se encuentra devuelve -1.
- **lastIndexOf(Object o)**: devuelve la posición de la última vez que un elemento coincida con el objeto pasado por parámetro. Si el elemento no se encuentra devuelve -1.
- **size()**: devuelve el número de elementos de la lista.
- **contains(Object o)**: devuelve verdadero si en la lista aparece el objeto pasado por parámetro, para lo cual utiliza intrínsecamente el método equals().

- **ArrayList** Es como un array que puede crecer dinámicamente. Ofrece rápida iteración y rápido acceso aleatorio. Es una colección ordenada pero no clasificada. Un **ArrayList** implementa el interfaz **RandomAccess**. Hay que elegirlo en vez de **LinkedList** cuando necesitemos iteración rápida y no vamos a necesitar mucha inserción y borrado
- **Vector** Es básicamente lo mismo que un **ArrayList**, pero los métodos de Vector son sincronizados para la seguridad de los hilos (threads). También implementa **RandomAccess**.
- **LinkedList** Está ordenada por la posición del índice, como el **ArrayList**, excepto por que los elementos están doblemente enlazados el uno al otro. Este enlazado da nuevos métodos para añadir y quitar desde el principio o el final, lo que le hace una buena elección para implementar una pila o una cola. Hay que tener en cuenta que **LinkedList puede iterar más lentamente que ArrayList, pero es una buena elección cuando queremos inserción y borrado rápidos.**

Interface Set

Un **Set** se preocupa de la unicidad (no permite duplicados). El método **equals()** determina si dos objetos son idénticos (en cuyo caso, sólo uno puede estar en el conjunto). Tiene tres implementaciones. Algunos métodos de la interface Set:

- **add(Object o)**: añade el objeto pasado por parámetro al Set siempre que éste no exista ya, y devuelve un booleano.
- **clear()**: Elimina a todos los elementos del Set.
- **contains(Object o)**: devuelve true si el Set contiene el objeto pasado por parámetro. Para ello, se compara de forma interna con el método equals (o.equals(x);) o con el método hashCode().
- **isEmpty()**: devuelve true si el Set está vacío.
- **iterator()**: devuelve un iterador
- **remove(Object o)**: elimina el objeto pasado por parámetro si existe y devuelve un booleano.
- **size()**: devuelve un entero que es el número de elementos del Set.

- **HashSet** Es un Set sin orden ni clasificación. Usa el **hashCode** del el objeto insertado, así que cuanto más eficiente sea la implementación del hashCode, mayor será el rendimiento. Lo usaremos cuando queramos una colección sin duplicados y no nos preocupe el orden cuando iteramos.
- **LinkedHashSet** Es una versión ordenada de HashSet que mantiene una lista doblemente enlazada a lo largo de todos sus elementos. Lo usaremos en vez de HashSet cuando nos preocupe el orden de iteración. Cuando iteramos a través de un HashSet el orden es impredecible, mientras que en un LinkedHashSet podemos iterar en el orden de inserción de los elementos.

Cuando usamos HashSet o LinkedHashSet, los objetos añadidos deben sobrescribir el método hashCode().

- **Treeset** Es una de las dos colecciones clasificadas (junto con TreeMap). Garantiza que todos los elementos estarán en orden ascendente, de acuerdo a su orden natural. Opcionalmente, podemos construir un TreeSet con un constructor que permite establecer nuestras propias reglas para la colección acerca de cuál debería ser el orden utilizando Comparator.

El **interface SortedSet** extiende al interfaz Set, y es implementado por TreeSet. Algunos de sus métodos:

- **comparator()**: obtiene el objeto pasado al constructor para establecer el orden; si se emplea el orden natural definido por la interface **Comparable**, devuelve null;
- **first() / last()**: devuelve el primer o el último elemento del conjunto.

Interface Map

Un mapa se preocupa de la existencia de identificadores únicos. Mapeamos una clave única a un valor específico, donde ambos, la clave y el valor, son objetos.

Las implementaciones de Map nos permiten hacer cosas como buscar un valor basándonos en una clave, pedir una colección de sólo las claves o pedir una colección de sólo los valores. Como los **Sets**, los **Maps** confían en el test **equals()** para saber si dos claves son iguales o diferentes. Algunos métodos de la interface **Map**:

- **clear()**: elimina todos los mapeos del Map.
- **containsKey(Object clave)**: devuelve true si el Map contiene un mapeo igual que el objeto pasado por parámetro:
boolean existe = productos.containsKey(producto);
- **containsValue(Object valor)**: devuelve true si el Map mapea a uno o más claves del objeto valor pasado por parámetro.
- **get(Object clave)**: devuelve el objeto valor de la clave pasada por parámetro; o null si el Map no encuentra ningún mapeo con esta clave.
- **isEmpty()**: devuelve true si el Map está vacío.

- **put(Clave clave, Valor valor):** asigna el valor pasado con la clave especificada a otro objeto valor.
- **remove(Object clave):** elimina el mapeo que tenga la clave pasada por parámetro si existe, devolviendo el valor de dicho mapeo.
- **size():** devuelve un entero con el número de mapeos del Map.

- **HashMap** Nos da un Map sin ordenar y sin clasificar. Cuando necesitamos un Map y nos preocupa el orden de iteración, el HashMap es apropiado. Donde se posicionan las claves en el Map está basado el hashcode de la clave, así que, como HashSet, cuanto más eficiente sea la implementación del método **hashcode()**, mejor rendimiento obtendremos. HashMap permite una clave null y múltiples valores **null** en una colección.
- **Hashtable** Es como un HashMap sincronizado. No se sincroniza una clase completa: Cuando decimos que Vector o Hashtable son sincronizados, nos referimos a que los métodos clave de esas clases son sincronizados. Otra diferencia es que aunque HashMap permite valores null así como una clave null, un Hashtable NO permite que nada sea null.
- **LinkedHashMap** Mantiene el orden de inserción (u opcionalmente el orden de acceso). Aunque es un poco más lenta que HashMap al añadir y eliminar elementos, se puede esperar una iteración más rápida con LinkedHashMap.
- **TreeMap** Es un Map clasificado (por el orden natural de sus elementos). Como TreeSet, TreeMap nos permite definir una clasificación a medida (vía un Comparator) cuando construimos un TreeMap que especifica cómo se deberían comparar entre sí los elementos cuando están siendo ordenados. TreeMap implementa a NavigableMap.

Interface Queue

Un Queue está diseñado para mantener una lista de cosas para ser procesadas de alguna forma. Aunque son posibles otros ordenamientos, las colas (queues) usan FIFO. Los Queues soportan todos los métodos estándar de Collection y también tienen métodos para añadir, eliminar y revisar los elementos de la cola. Algunos métodos del interface Queue:

- **offer(E e):** Inserta el elemento especificado en la cola. Si es posible devuelve true, en caso contrario devuelve false.
- **peek():** Recupera, pero no lo elimina, el elemento a la cabeza de la cola. Devuelve null si la cola no tiene elementos.
- **poll():** Recupera y elimina, el elemento a la cabeza de la cola. Devuelve null si la cola no tiene elementos.
- **remove():** Recupera y elimina la cabeza de la cola.

PriorityQueue El propósito de la clase **PriorityQueue** es crear una cola "priority in - priority out" en vez de la clásica cola FIFO. Los elementos de una PriorityQueue están ordenados por orden natural (en cuyo caso los elementos que están clasificados primero serán accedidos primero) o según un Comparador. En los dos casos el orden de los elementos representa su prioridad relativa.

ArrayDeque Es una cola con terminación doble. Esto significa que podemos añadir y quitar elementos de ambos extremos de la cola. Es una buena elección para implementar tanto una cola como una pila debido a que es de tamaño variable sin capacidad para restricciones y está diseñada para alto rendimiento (aunque no es thread safe).

Class	Map	Set	List	Ordered	Sorted
HashMap	X			No	No
Hashtable	X			No	No
TreeMap	X			Sorted	By <i>natural order</i> or custom comparison rules
LinkedHashMap	X			By insertion order or last access order	No
HashSet		X		No	No
TreeSet		X		Sorted	By <i>natural order</i> or custom comparison rules
LinkedHashSet		X		By insertion order	No
ArrayList			X	By index	No
Vector			X	By index	No
LinkedList			X	By index	No
PriorityQueue				Sorted	By to-do order
ArrayDeque				By position	No

Usando Collections

Usando ArrayList

Algunas de sus ventajas de **ArrayList** sobre los arrays normales son:

- Puede crecer dinámicamente.
- Proporciona mecanismos de inserción y búsqueda más potentes que los de las matrices.

Tomamos como ejemplo un array que contiene strings. En la práctica lo instanciaremos polimórficamente:

```
List miLista = new ArrayList( )
```

Desde Java 5, la forma correcta de declarar el tipo de una colección es usando genéricos (<**String**>):

```
List<String> miLista = new ArrayList<String>( )
```

Algunas capacidades de ArrayList:

```
List<String> test = new ArrayList<String>( );
String s ="hi";
test.add("string");
test.add(s);
test.add(s+s);
System.out.println(test.size( ));
System.out.println(test.contains(42));
System.out.println(test.contains("hihi"));
test.remove("hi");
System.out.println(test.size( ));
```

Autoboxing con Colecciones

En general, las colecciones pueden contener objetos, pero no primitivas. Sin embargo si asignamos directamente una primitiva a una colección el mecanismo de **autoboxing** la envuelve en su clase correspondiente por nosotros:

```
List miLista = new ArrayList( );
miLista.add(43); // Por debajo hace miLista.add(new Integer(43))
```

Boxing, ==, y equals()

Echemos un vistazo a cómo funcionan las envolturas (wrappers) con **==**, **!=** y **equals()**. Los desarrolladores de la API decidieron para todas las clases de envoltura, que dos objetos son iguales si son del mismo tipo y tienen el mismo valor.

```
Integer i1 = 1000;
Integer i2 = 1000;
if(i1 != i2) System.out.println("Objetos diferentes");
if(i1.equals(i2)) System.out.println("Significativamente iguales.");
```

Debido a que tienen el mismo valor int, el método **equals()** los considera "significativamente equivalentes", y en consecuencia devuelve true. Y ahora:

```
Integer i3 = 10;
Integer i4 = 10;
if(i3 == i4) System.out.println("Mismo objeto");
if(i3.equals(i4)) System.out.println("Significativamente iguales.");
```

El método **equals()** parece funcionar, pero ¿qué ocurre con **==** y **!=**? Si ejecutamos los ejemplos anteriores, **!=** nos dice que i1 e i2 son objetos diferentes cuando **==** nos dice i3 e i4 son el mismo objeto!!! Esto es debido a que dos instancias de las siguientes clases de envoltura (creados a través de boxing) siempre serán **==** cuando sus valores primitivos sean iguales:

- Boolean
- Byte
- Character from \u0000 a \u007f (7f es 127 en decimal).
- Short e Integer de -128 a 127.

Cuando se usa **==** para comparar una primitiva con un envuelto, el envuelto será desenvuelto y la comparación será primitiva a primitiva.

Donde puede ser usado el boxing

Siempre que queramos que nuestra colección albergue objetos y primitivas, querremos usar envolturas para hacer a las primitivas compatibles con la colección. La regla general es que boxing y unboxing funcionan dondequiera que normalmente podemos usar una primitiva o un objeto envuelto. Lo siguiente es legal:

```
class UseBoxing {
    public static void main(String[ ] args){
        UseBoxing u = new UseBoxing( );
        u.go(5);
    }
}
```



```
boolean go(Integer i) {
    Boolean ifSo = true;
    Short s = 300;
    if(ifSo){
        System.out.println(++s);
    }
    return !ifSo;
}
}
```

La notación “diamante” de Java 7

La notación diamante se ve al final del tema. Previo a Java 7:

```
ArrayList<String> stuff = new ArrayList<String>( );
List<Dog> myDogs = new ArrayList<Dog>( );
Map<String,Dog> dofMap = new HashMap<String, Dog>( );
```

Desde Java 7, la notación se ha simplificado:

```
ArrayList<String> stuff = new ArrayList< >( );
List<Dog> myDogs = new ArrayList< >( );
Map<String,Dog> dofMap = new HashMap< >( );
```

La siguiente declaración NO es legal:

```
List< > stuff = new ArrayList<String>( );
```

Clasificando Colecciones y Arrays

Tanto colecciones como arrays pueden ser clasificados y buscados usando métodos de la API.

- **Clasificando colecciones**

Por ejemplo, hay un par de maneras de clasificar un ArrayList. Por ahora usaremos la clase `java.util.Collections`, más adelante usaremos el método `sort( )` de `ArrayList`:

```
import java.util.*;
class TestSort1 {
    public static void main(String[ ] args){
        ArrayList<String> stuff = new ArrayList<String>( );
        stuff.add("Denver");
        stuff.add("Vail");
        stuff.add("Boulder");
```

```

stuff.add("Aspen");
stuff.add("Telluride");
System.out.println("Sin clasificar " + stuff);
Collections.sort(stuff); // Lo clasifica alfabéticamente
System.out.println("Clasificado " +stuff);
}
}

```

Imaginemos ahora que estamos desarrollando una aplicación para controlar nuestros DVD. Ya tenemos creado el software para gestionar el I/O hacia el archivo **dvdInfo.txt**.

```

class DVDInfo {
    String title;
    String genre;
    String leadActor;
    DVDInfo(String t, String g, String a) {
        title = t; genre = g; leadActor = a;
    }
    public String toString( ){
        return title + " " + genre + " " + leadActor + "\n";
    }
    // Aquí getters y setters.
}

```

Aquí la información almacenada en **dvdinfo.txt**:

```

Donnie Darko/sci-fi/Gyllenhall, Jake
Raiders of the Lost Ark(action/Ford, Harrison
2001/sci-fi/??
Caddyshack/comedy/Murray, Bill
Star Wars/sci-fi/Ford, Harrison
Lost in Translation/comedy/Murray, Bill
Patriot Games/action/Ford, Harrison

```

En nuestra aplicación queremos crear una instancia de DVDInfo para cada línea de datos que leemos del archivo. Para cada instancia, procesaremos la línea de datos y daremos valor a tres variables de instancia de DVDInfo. Finalmente, queremos poner todas las instancias de DVDInfo en un ArrayList. Imaginemos un método populateList() que hace todo esto.

```

ArrayList<DVDInfo> dvdList = new ArrayList<DVDInfo>( );
populateList( );
System.out.println(dvdList);

```

Podemos intentar clasificarlo con:

```
Collections.sort(dvdList;)
```

Pero **NO** funciona, no compilará. ¿Por qué? Porque no hemos definido ningún criterio para poder comparar objetos de la clase DVDInfo. Para poder hacer esto, la clase de los objetos de la lista debería implementar el interface **Comparable**.

Por el contrario, podemos clasificar un ArrayList de Strings alfabéticamente sin problema:

```
ArrayList<String> miLista = new ArrayList<String>( );  
Collections.sort(miLista);
```

La clase String ya implementa **Comparable**, por eso podemos usar el método **sort()**.

El interface Comparable

El interface **Comparable** es utilizado por el método **Collections.sort()** y el método **java.util.Arrays.sort()** para clasificar List y matrices de objetos, respectivamente. Para implementar el interface **Comparable**, una clase debe implementar un único método:

```
public int compareTo(tipo a ordenar)
```

Este método retorna un entero con las siguientes características:

```
int x= unObjeto.compareTo(otroObjeto);
```

- **Negativo** Si unObjeto < otroObjeto.
- **Cero** Si unObjeto = otroObjeto
- **Positivo** Si unObjeto > otroObjeto.

El método **sort()** usa **compareTo()** para determinar cómo deberá ser clasificada List o el array. Queda a nuestro criterio establecer el contenido de **compareTo()**. Por ejemplo:

```
class DVDInfo implements Comparable<DVDInfo>{  
    // Código existente  
    public int compareTo(DVDInfo d){  
        return title.compareTo(d.getTitle( )); // Comparamos los títulos (strings)  
    }  
}
```

Es importante recordar que cuando se sobreescribe **equals()** SE DEBE tomar un argumento de tipo **Object**, pero cuando se sobreescribe **compareTo()** deberemos tomar un argumento del tipo que estamos clasificando.

Poniendo todo junto:

```

class DVDInfo implements Comparable<DVDInfo> {
    String title;
    String genre;
    String leadActor;
    DVDInfo(String t, String g, String a) {
        title = t; genre = g; leadActor = a;
    }
    public String toString() {
        return title + " " + genre + " " + leadActor + "\n";
    }
    public int compareTo(DVDInfo d){
        return title.compareTo(d.getTitle( ));
    }
    public String getTitle() {
        return title;
    }
    // Otros getters y setters.
}

```

Desgraciadamente, podemos implementar **Comparable** una única vez en la clase, por ello sólo podremos especificar un único criterio de clasificación de esta manera. ¿Qué hacemos si eventualmente necesitamos aplicar un criterio de clasificación distinto? Además, se nos obliga a alterar la clase implementando un interface y desarrollando un método. Afortunadamente tenemos una alternativa que viene a continuación.

Clasificando con Comparator

El interfaz **Comparator** nos da la capacidad de clasificar una colección dada en un número de diferentes formas. Además, a diferencia del interfaz **Comparable**, que nos fuerza a cambiar la clase cuyas instancias deseamos clasificar, el interfaz **Comparator** se puede usar para clasificar instancias de cualquier clase (incluso clases que no pueden ser modificadas). Este interface tiene un único método:

```

public int compare(tipo a ordenar)

```

```

import java.util.*;
class GenreSort implements Comparator<DVDInfo>{
    public int compare(DVDInfo one,DVDInfo two) {
        return one.getGenre( ).compareTo(two.getGenre( ));
    }
}

```

El método **Comparator.compare()** devuelve un int cuyo significado es el mismo que el retorno de **Comparable.compareTo()**. Para comparar **Comparable** con **Comparator**:

```

import java.util.*;
import java.io.*;

```

```

public class TestDVD {
    ArrayList<DVDInfo> dvdlist = new ArrayList<DVDInfo>( );
    public static void main(String[ ] args){
        new TestDVD( ).go( );
    }
    public void go( ){
        populateList( );
        System.out.println(dvdlist);
        Collections.sort(dvdlist);
        System.out.println(dvdlist);

        GenreSort gs = new GenreSort( );
        Collections.sort(dvdlist, gs);
        System.out.println(dvdlist);
    }
    public void populateList( ){
        // Leer del archive y llenar el ArrayList
    }
}

```

java.lang.Comparable	java.util.Comparator
int objOne.compareTo(objTwo)	int compare(objOne, objTwo)
Returns negative if objOne < objTwo zero if objOne == objTwo positive if objOne > objTwo	Same as Comparable
You must modify the class whose instances you want to sort.	You build a class separate from the class whose instances you want to sort.
Only one sort sequence can be created.	Many sort sequences can be created.
Implemented frequently in the API by: String, wrapper classes, LocalDate, LocalTime...	Meant to be implemented to sort instances of third-party classes.

Clasificando ArrayLists usando el método sort()

Los **ArrayList** también pueden ser clasificados directamente invocando al método **sort()** en la lista. Al igual que **Collections.sort()**, el método **ArrayList.sort()** usa un **Comparator**. Así que podemos pasar una instancia de una clase que implementa **Comparator**:

```

GenreSort gs = new GenreSort( );
dvdlist.sort(gs); // sort usando el comparator GenreSort

```

O podemos pasar el Comparator hecho con la **expresión lambda**:

```
// Clasificar por género, usando un comparador lambda
Dvdlist.sort((one,two) -> one.getGenre( ).compareTo(two.getGenre( )));
```

De las dos formas se obtiene el mismo resultado.

Clasificando con la clase Arrays

Ahora atendamos a utilizar la clase **java.util.Arrays** para clasificar arrays. Clasificar arrays de objetos es igual que clasificar colecciones de objetos. El método **Arrays.sort()** está sobrescrito de la misma manera que lo está **Collections.sort()**:

- **Arrays.sort (arrayToSort)**
- **Arrays.sort (arrayToSort, Comparator)**

Los métodos **Arrays.sort(myArray)** que ordenan arrays de primitivas siempre ordenan basándose en el orden natural .

Los métodos **sort()** tanto de Collections como de Arrays son **static**, y alteran los objetos que están ordenando, en vez de devolver una copia ordenada en otro objeto. También hay que tener cuidado: Para poder ordenar de cualquier forma una colección o un array sus elementos deben ser **mutuamente comparables**.

Buscando en Arrays y Collections

Las clases Collections y Arrays proporcionan métodos Para permitir buscar un elemento específico. Para buscar en colecciones o en arrays, se aplican las siguientes reglas:

- Las búsquedas se realizan usando el método **binarySearch()**.
- Las búsquedas con éxito devuelven el índice entero del elemento buscado.
- Las búsquedas **sin éxito** devuelven un entero que representa el **punto de inserción**. El punto de inserción es el lugar en la colección o el array en donde el elemento sería insertado para mantener la colección o el array adecuadamente clasificado. El método **binarySearch()** usa números negativos para indicar puntos de inserción. Como 0 es un resultado válido para una inserción con éxito, el primer punto de inserción disponible es el -1. Por ejemplo, si el punto de inserción de una búsqueda es el elemento 2, el punto de inserción retornado sería -3.
- La colección o el array en el que se busca DEBE estar clasificado antes de poder buscar, en caso contrario el resultado de la búsqueda será impredecible.

- Si la colección o el array en el que estamos buscando está ordenado por su **orden natural**, deberemos buscar en él en orden natural (normalmente esto se consigue **NO** usando un Comparator como argumento del método **binarySearch()**).
- Si la colección o el array fue ordenado usando un Comparator, **SE DEBE** buscar usando el mismo Comparator, que se pasará como tercer argumento del método **binarySearch()**. Los Comparator **NO** pueden ser usados cuando buscamos en arrays de primitivas.

```
import java.util.*;
class SearchObjArray {
    public static void main(String[] args){
        String[] sa = {"one","two","three","four"};
        Arrays.sort(sa);           // Orden natural
        for(String s : sa)
            System.out.println(s+" ");
        System.out.println("uno = " + Arrays.binarySearch(sa, "one"));
        System.out.println("Ahora en orden inverso");
        ResortComparator rs = new ResortComparator( );
        Arrays.sort(sa,rs); // Ordenado con Comparator
        for(String s : sa)
            System.out.println(s+" ");
        System.out.println("uno = " + Arrays.binarySearch(sa, "one")); // Incorrecto
        System.out.println("uno = " + Arrays.binarySearch(sa, "one", rs));
    }
    static class ResortComparator implements Comparator<String> {
        public int compare(String a, String b){
            return b.compareTo(a);
        }
    }
}
```

Convirtiendo Arrays a Lists y Lists a Arrays

Un par de métodos permiten convertir un array a List y a la inversa. Las clases **List** y **Set** tienen métodos **toArray()**, y la clase **Arrays** tiene un método llamado **asList()**.

El método **Arrays.asList()** copia un array en un List. Cuando se usa el método **asList()**, el array y el List se unen. Cuando actualizamos uno, el otro se actualiza automáticamente:

```
String[] sa = {"one","two","three","four"};
List sList = Arrays.asList(sa);
System.out.println("tamaño " + sList.size( ));
System.out.println("idx2 " + sList.get(2));
```

```
sList.set(3,"six");
sa[1] = "five";
for (String s : sa)
System.out.println(s + " ");
System.out.println("sl[1] " + sList.get(1));
```

El método `toArray( )` viene en dos sabores, uno que retorna un nuevo array de objetos y otro que usa un array para enviarlo al array de destino:

```
List<Integer> iL = new ArrayList<Integer>( );
for(int x=0; x<3; x++)
    iL.add(x);
Object[ ] oa = iL.toArray( ); // Crea un array d objetos
Integer[ ] ia2 = new Integer[3];
ia2 = iL.toArray(ia2); // Crea una array de Integer
```

Usando Lists

Las listas son normalmente usadas para guardar las cosas con alguna clase de orden, permitiéndose duplicados. Antes de Java 5 y el `for` mejorado, la forma más común de recorrer un `List` elemento a elemento era usando un **Iterator**. Un **Iterator** es un objeto que está asociado específicamente a una colección. Nos permite iterar uno por uno a través de los elementos de una colección. Sus métodos más importantes:

- **boolean hasNext()** Devuelve `true` si hay al menos un elemento más en la colección siendo recorrida. Invocando a este método NO nos desplazamos al siguiente elemento de la colección.
- **Object next()**: Retorna el siguiente elemento en la colección, y nos mueve al siguiente elemento.

Se usa:

```
List<Persona> per = new ArrayList<Persona>( );
Iterator<Persona> i1 = per.iterator( );
while (i1.hasNext( )){
    Persona p = i1.next( );
}
```

Otro ejemplo:

```
import java.util.*;
class Dog {
    public String name;
    Dog(String n){ name = n; }
```



```

}
class ItTest {
    public static void main(String[ ] args){
        List<Dog> d = new ArrayList<Dog>( );
        Dog dog = new Dog("aiko");
        d.add(dog);
        d.add(new Dog("clover"));
        d.add(new Dog("magnolia"));
        Iterator<Dog> i3 = d.iterator( );
        while (i3.hasNext( )) {
            Dog d2 = i3.next( );
            System.out.println(d2.name);
        }
        System.out.println("tamaño " + d.size);
        System.out.println("get1 " + d.get(1).name);
        System.out.println("aiko " + d.indexOf(dog));
        d.remove(2);
        Object[ ] oa = d.toArray( );
        for(Object o : oa){
            Dog d2 = (Dog)o;
            System.out.println("oa " + d2.name);
        }
    }
}

```

Hemos usado genéricos para crear un **Iterator**. Gracias a esto no tenemos que hacer un casting sobre el Object devuelto por next(). Podríamos haber declarado el **Iterator** sin genéricos, pero tendríamos que hacer un casting:

```

Iterator i3 = d.iterator( );
while (i3.hasNext( )) {
    Dog d2 = (Dog) i3.next( );
    System.out.println(d2.name);
}

```

Usando Sets

Los Sets se usan cuando no queremos duplicados en una colección. Si intentamos añadir a un Set un elemento que ya existe en el Set, el elemento duplicado no será añadido, y el método add() retornará false.

```

import java.util.*;
class SetTest {
    boolean[ ] ba = new boolean[5];
    Set s = new HashSet( ); // new LinkedHashSet( );
    ba[0] = s.add("a");
}

```

```

ba[1] = s.add("new Integer("56");
ba[2] = s.add("b");
ba[3] = s.add("a"); // Falla: Está duplicado
ba[4] = s.add(new Object( ));
for (int x=0; x<ba.length; x++)
    System.out.println(ba[x] + " ");
System.out.println("\n");
for (Object o: s)
    System.out.println(o + " ");
}
}

```

Compila y se ejecuta, sin embargo el orden del segundo for no es predecible: Los **HashSets** y los **LinkedHashSets** no garantizan ningún orden. Si usamos un **TreeSet**:

```

import java.util.*;
class SetTest {
    boolean[ ] ba = new boolean[5];
    Set s = new TreeSet( );
    ba[0] = s.add("a");
    ba[1] = s.add("new Integer("56");
    ba[2] = s.add("b");
    ba[3] = s.add("a");
    ba[4] = s.add(new Object( ));

```

```

    for (int x=0; x<ba.length; x++)
        System.out.println(ba[x] + " ");
    System.out.println("\n");
    for (Object o: s)
        System.out.println(o + " ");
}
}

```

No compilará. **TreeSet es sorted**, por lo que es obligado que sus elementos sean mutuamente comparables. A menos que lo especifiquemos de otra manera usando **Comparable** o **Comparator**, los objetos de tipos diferentes no son mutuamente comparables.

Usando Maps

Cuando usamos una clase que implementa **Map**, cualquier clase que se use como una parte de las claves de ese mapa, debemos sobrescribir los métodos **hashCode()** y **equals()**. En realidad no es obligatorio, sólo tendremos que sobrescribir en caso de que deseemos recuperar datos de nuestro mapa. Sino los sobreescribimos simplemente no podremos buscar).

```

import java.util.*;
class Persona {
    public String nombre;
    public Persona(String n) { nombre = n; }
    public boolean equals(Object o)
    {
        if((o instanceof Persona && (((Persona)o).nombre == nombre)
            return true;
        else
            return false;
    }
    public int hashCode( ) { return nombre.length( ); }

    class Animal{ }

    enum Mascota {PERRO, GATO, CABALLO}

class MapTest{
    Map<Object,Object> m = new HashMap<Object, Object>( );
    m.put("k1", new Persona("Ernesto"));
    m.put("k2", Mascota.PERRO);
    m.put(Mascota.GATO, "Clave GATO");
    Persona p1 = new Persona("Rodrigo");
    m.put(p1, "Clave Persona");
    m.put(new Animal( ), "Clave Animal");

    System.out.println(m.get("k1"));
    String k2 ="k2";
    System.out.println(m.get(k2));
    Mascota m = Mascota.GATO;
    System.out.println(m.get(m));
    System.out.println(m.get(p1));
    System.out.println(m.get(new Animal( )));
    System.out.println(m.size( ));
    }
}

```

En el ejemplo podemos usar una enumeración como clave debido a que la clase **java.lang.Enum** sobrescribe los métodos **equals()** y **hashCode()**. Al ejecutar comprobamos que como la clase **Animal** no sobrescribe estos dos métodos, obtenemos un nulll.

Por otra parte ¿Qué sucede cuando se producen cambios en las variables de un objeto que es usado como clave?

```

p1.name ="magnolio";
System.out.println(m.get(p1));

```

Cuidado cuando cambiamos alguna propiedad de un objeto usado como clave: Si la propiedad es usada en el método **hashCode()**, al cambiar el valor de la propiedad cambia el valor del **hashCode()** y la recuperación del objeto mapeado no sería posible.

Navegando (buscando) en TreeSet y TreeMap

Imaginemos que estamos haciendo una aplicación para gestionar una estación de tren con unos horarios irregulares. El siguiente código determina dos cosas:

1. El último tren que parte antes de la 4 pm (16:00).
2. El primer tren que parte después de las 8 pm (20:00)

```
import java.util.*;
public class Ferry{
public static void main(String[ ] args){
TreeSet<Integer> times = new TreeSet<Integer>( );
times.add(1205); // Añadimos algunas horas de salida
times.add(1505);
times.add(1545);
times.add(1830);
times.add(2010);
times.add(2100);
```

```
// Versión Java 5
```

```
TreeSet<Integer> subset = new TreeSet<Integer>( );
Subset = (TreeSet)times.headSet(1600);
System.out.println("Java 5 – El último antes de las 4 pm es: " + subset.last( ));
TreeSet<Integer> sub2 = new TreeSet<Integer>( );
sub2 = (TreeSet)times.tailSet(2000);
System.out.println("Java 5 – El último después de las 8 pm es: " + sub2.first( ));
```

```
// A partir de Java 6
```

```
System.out.println("Java 6 – El último antes de las 4 pm es: " + times.lower(1600));
System.out.println("Java 6 – El último después de las 8 pm es: " + times.higher(2000));
}
}
```

A continuación una tabla con los principales métodos de navegación:

Method	Description
<code>TreeSet.ceiling(e)</code>	Returns the lowest element $\geq e$
<code>TreeMap.ceilingKey(key)</code>	Returns the lowest key $\geq key$
<code>TreeSet.higher(e)</code>	Returns the lowest element $> e$
<code>TreeMap.higherKey(key)</code>	Returns the lowest key $> key$
<code>TreeSet.floor(e)</code>	Returns the highest element $\leq e$
<code>TreeMap.floorKey(key)</code>	Returns the highest key $\leq key$
<code>TreeSet.lower(e)</code>	Returns the highest element $< e$
<code>TreeMap.lowerKey(key)</code>	Returns the highest key $< key$
<code>TreeSet.pollFirst()</code>	Returns and removes the first entry
<code>TreeMap.pollFirstEntry()</code>	Returns and removes the first key/value pair
<code>TreeSet.pollLast()</code>	Returns and removes the last entry
<code>TreeMap.pollLastEntry()</code>	Returns and removes the last key/value pair
<code>TreeSet.descendingSet()</code>	Returns a <code>NavigableSet</code> in reverse order
<code>TreeMap.descendingMap()</code>	Returns a <code>NavigableMap</code> in reverse order

- **Polling (sondeo)**

La idea del sondeo es que queremos recuperar y eliminar un elemento desde el principio o el final de una colección. En el caso de **TreeSet**, **pollFirst()** devuelve y elimina la primera entrada en el set, y **pollLast()** devuelve y elimina la última. De forma similar **TreeMap** proporciona **pollFirstEntry()** y **pollLastEntry()** para obtener y eliminar pares clave/valor.

- **Orden descendente**

Java 6 también añadió para **TreeSet** y **TreeMap** métodos que retornan una colección en orden inverso, como **TreeSet.descendingSet()** y **TreeMap.descendingMap()**.

Colecciones respaldadas

Algunas de las clases del paquete **java.util** soportan el concepto de “colecciones respaldadas (**backed collections**)”.

```
TreeMap<String, String> map = new TreeMap<String, String>( );
map.put("a","ant");
map.put("d","dog");
map.put("h","horse");
SortedMap<String, String> submap;
submap = map.subMap("b","g");    // Se crea la backed collection
System.out.println(map + " " + submap);    // Muestra contenidos
```

```
map.put("b","bat");    // Se añade al original
submap.put("f","fish"); //Se añade a la copia
map.put("b","racoon"); // Se añade al original (fuera de rango)
// submap.put("f","fish"); // Se añade a la copia (fuera de rango)
System.out.println(map + " " + submap); // Muestra el contenido final
```

En el ejemplo anterior, el método **subMap()** está haciendo una copia de una porción del **TreeMap** llamado map. Cuando añadimos pares clave/valor en el **TreeMap** original o en la copia parcial **SortedMap**, las nuevas entradas fueron añadidas a la otra colección de forma automática (no siempre). Cuando se creó submap, se proporcionó un rango de valores para la nueva colección. Este rango define, no sólo que debería ser incluido cuando la copia parcial es creada, sino que también define el rango de valores que puede ser añadido a la copia. Si añadimos nuevas entradas fuera del rango en el original, no se mostrarán en la copia, y si intentamos añadir un valor fuera de rango a la copia, se lanzará una excepción. A continuación una tabla con los métodos “backed collection” para **TreeSet** y **TreeMap**:

Method	Description
headSet(e, b*)	Returns a subset ending at element e and <i>exclusive</i> of e
headMap(k, b*)	Returns a submap ending at key k and <i>exclusive</i> of key k
tailSet(e, b*)	Returns a subset starting at and <i>inclusive</i> of element e
tailMap(k, b*)	Returns a submap starting at and <i>inclusive</i> of key k
subSet(s, b*, e, b*)	Returns a subset starting at element s and ending just before element e
subMap(s, b*, e, b*)	Returns a submap starting at key s and ending just before key e

El * indica argumentos boolean opcionales.

PriorityQueue

A diferencia de colas que son **FIFO** por defecto, **PriorityQueue** ordena sus elementos usando una prioridad definida por el usuario. La prioridad puede ser tan simple como el orden natural (ya establecido por defecto en **String** y clases de envoltura), pero podemos establecer otro usando **Comparator**. Además, las **Queue** tienen métodos propios que no podemos encontrar en otros interfaces: **peek()**, **poll()**, **offer()**.

```
import java.util.*;
class PQ {
    static class PQsort implements Comparator<Integer>{
        public int compare( Integer uno, Integer dos){
            return dos-uno;    // Unboxing
        }
        public static void main (String[ ] args)
        {
            int[ ] ia = {1,5,6,7,3,2,5};    // Datos desordenados
```

```

PriorityQueue<Integer> pq1 = new PriorityQueue<Integer>( ); // Orden natural

for(int x : ia)    // Cargar cola
    pq1.offer(x);
for(int x : ia)    // Revisar cola
    System.out.print(pq1.poll( ) + " ");
System.out.println(" ");

PQsort pqs = new PQsort( );    // Tomamos un Comparator
PriorityQueue<Integer> pq2 = new PriorityQueue<Integer>(10,pqs);

for(int x : ia)    // Cargar cola
    pq2.offer(x);
System.out.println( "size " + pq2.size( ));
System.out.println( "peek " + pq2.peek( ));
System.out.println( "size " + pq2.size( ));
System.out.println( "poll " + pq2.poll( ));
System.out.println( "size " + pq2.size( ));

for(int x : ia)    // Revisar cola
    System.out.println(pq2.poll( ) + " ");
}
}

```

ArrayDeque

ArrayDeque es una clase Collection que implementa el interfaz **Deque**. Este es un interfaz para colas doblemente terminadas, con métodos para añadir y eliminar elementos a y desde la cola en el otro extremo.

La mayor ventaja de **ArrayDeque** sobre, por ejemplo, un **List** es el rendimiento. A diferencia de **PriorityQueue** no hay ordenamiento natural en **ArrayDeque**; es simplemente una colección de elementos que están clasificados en el orden en que fueron añadidos.

```

List<Integer> nums = Arrays.asList(10, 9 ,8 ,7 ,6 ,5);
// Crea varios ArrayDeques, cada uno con espacio para dos items.
ArrayDeque<Integer> a = new ArrayDeque<>(2);
ArrayDeque<Integer> b = new ArrayDeque<>(2);
ArrayDeque<Integer> c = new ArrayDeque<>(2);
ArrayDeque<Integer> d = new ArrayDeque<>(2);
ArrayDeque<Integer> e = new ArrayDeque<>(2);
for(Integer n : nums) { // Añadir 6 items a cada Deque, usando diferentes métodos

```

```
a.offer(n);      // Añade al final
b.offerFirst(n)  // Añade al principio
c.push(n);       // Añade al principio
d.add(n);        // Añade al final
e.addFirst(n);   // Añade al principio
}
// Muestra las colas
System.out.println("a" + a);
System.out.println("b" + b);
System.out.println("c" + c);
System.out.println("d" + d);
System.out.println("e" + e);
```

Aunque hemos creado dos espacios, hay que recordar que el **ArrayDeque** no tiene restricción de tamaño, así que en el bucle for, en donde iteramos a través de la List de números y añadimos seis números a cada una de las **ArrayDeque**, no se producen excepciones ni errores. Podemos hacer algunas pruebas usando el ArrayDeque e:

```
// peek( ) Retorna sin eliminarlo el primer elemento de la cola.
System.out.println("Primer elemento de e: " + e.peek( ));
System.out.println(" e no ha cambiado: " +e);
```

```
// poll( ) Retorna y elimina el primer elemento de la cola.
System.out.println("Primer elemento de e: " + e.poll( ));
System.out.println(" e ha cambiado: " +e);
```

```
// pop( ) Retorna y elimina el primer elemento de la cola.
System.out.println("Primer elemento de e: " + e.pop( ));
System.out.println(" e ha cambiado: " +e);
```

```
// pollLast( ) Retorna y elimina el último elemento de la cola.
System.out.println("Primer elemento de e: " + e.pollLast( ));
System.out.println(" e ha cambiado: " +e);
```

```
// removeLast( ) Elimina el último elemento de la cola.
System.out.println("Primer elemento de e: " + e.removeLast( ));
System.out.println(" e ha cambiado: " +e);
```

Hay que recordar que a diferencia de muchas clases Deque, ArrayDeque no es thread safe.

Visión general a los métodos para Arrays y Collections

Key Methods in java.util.Arrays	Descriptions
static List asList (T[])	Convert an array to a List (and bind them).
static int binarySearch (Object[], key) static int binarySearch (primitive[], key)	Search a sorted array for a given value; return an index or insertion point.
static int binarySearch (T[], key, Comparator)	Search a Comparator-sorted array for a value.
static boolean equals (Object[], Object[]) static boolean equals (primitive[], primitive[])	Compare two arrays to determine if their contents are equal.
static void sort (Object[]) static void sort (primitive[])	Sort the elements of an array by natural order.
static void sort (T[], Comparator)	Sort the elements of an array using a Comparator.
static String toString (Object[]) static String toString (primitive[])	Create a String containing the contents of an array.
Key Methods in java.util.Collections	Descriptions
static int binarySearch (List, key) static int binarySearch (List, key, Comparator)	Search a "sorted" List for a given value; return an index or insertion point.
static void reverse (List)	Reverse the order of elements in a List.
static Comparator reverseOrder () static Comparator reverseOrder (Comparator)	Return a Comparator that sorts the reverse of the collection's current sort sequence.
static void sort (List) static void sort (List, Comparator)	Sort a List either by natural order or by a Comparator.

Visión general a los métodos para List, Set, Map y Queue

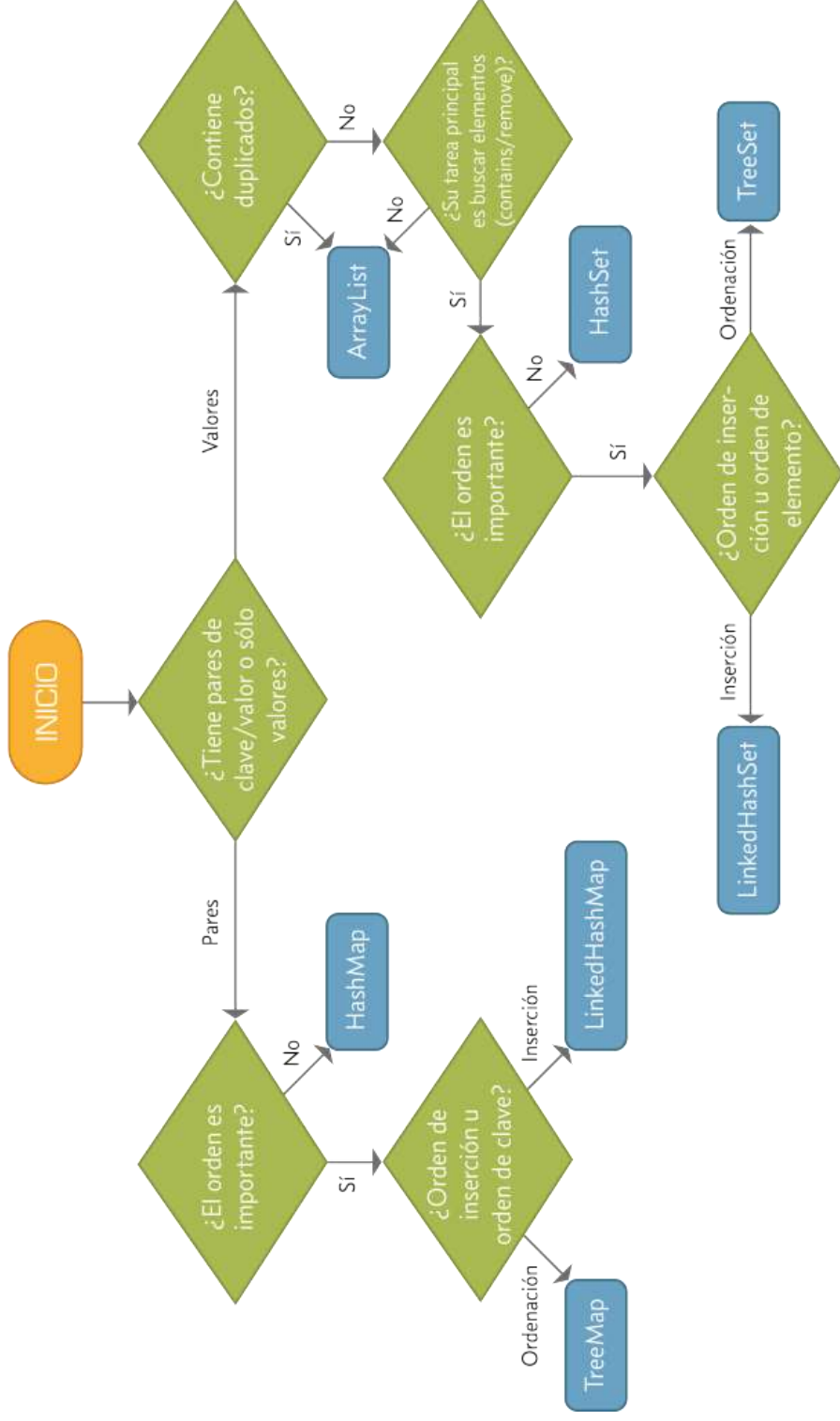
Key Interface Methods	List	Set	Map	Descriptions
boolean add {element} boolean add {index, element}	X X	X		Add an element. For Lists, optionally add the element at an index point.
boolean contains {object} boolean containsKey {object key} boolean containsValue {object value}	X	X	X X	Search a collection for an object (or, optionally for Maps, a key); return the result as a boolean.
object get {index} object get {key}	X		X	Get an object from a collection via an index or a key.
int indexOf {object}	X			Get the location of an object in a List.
Iterator iterator ()	X	X		Get an Iterator for a List or a Set.
Set keySet ()			X	Return a Set containing a Map's keys.
put {key, value}			X	Add a key/value pair to a Map.
element remove {index} element remove {object} element remove {key}	X X	X	X	Remove an element via an index, or via the element's value, or via a key.
int size ()	X	X	X	Return the number of elements in a collection.
Object[] toArray () T[] toArray {T[]}	X	X		Return an array containing the elements of the collection.

Es importante saber algunos detalles sobre el orden natural. El siguiente código ayudará a comprender las posiciones relativas de mayúsculas, minúsculas y espacios en un orden natural:

```
String[] sa = {">ff<", ">f<", ">f <", ">FF<"};
PriorityQueue<String> pq3 = new PriorityQueue<String>( );
for(String s : sa)
    pq3.offer(s);
for(String s : sa)
    System.out.println(pq3.poll( ) + " ");
```

Los espacios van delante de los caracteres y las mayúsculas delante de las minúsculas.

Diagrama de decisión para uso de colecciones Java



Genéricos

Los arrays en Java siempre han sido de tipo seguro. Un array declarado como de tipo String:

```
String[ ] cadenas = new String[ ];
```

sólo admitirá Strings, y no podrá aceptar otra cosa, como Integers por ejemplo. Sin embargo las colecciones no podían ser de tipo seguro antes de Java 5. Esto suponía un problema:

```
ArrayList myList = new ArrayList( );
```

O su polimórfico equivalente:

```
List myList = new ArrayList( );
```

No había una sintaxis para especificar que el objeto tomará Strings y sólo Strings. En realidad con esa sintaxis podría tomar cualquier tipo de objeto. Desde Java 5 podemos usar genéricos y, aunque no sirven sólo para hacer colecciones de tipo seguro, es para lo que más se usan.

La forma heredada de hacer Colecciones

Previo a Java 5, no se podía declarar el tipo:

```
List myList = new ArrayList( );  
myList.add("Fred"); // OK, contendrá Strings  
myList.add(new Dog( )); // También guardará instancias de Dog  
myList.add(new Integer(43)); // Y Integers...
```

Una colección sin usar genéricos puede contener cualquier tipo de objeto, **siempre que no sea una primitiva**. Por ello los métodos que extraen objetos de la colección no pueden tener más que un tipo de retorno: **java.lang.Object**

Esto significa que para poder recuperarlos en su clase original sería necesario hacer un casting:

```
String s = (String) myList.get(0);
```

Con genéricos:

```
List<String> miLista = new ArrayList<String>( );  
miLista.add("Pepe"); // OK, contendrá Strings  
miLista.add(new Animal( )); // Error al compilar!!
```

De esta manera controlamos lo que se guarda en la colección, y por tanto lo que puede salir de ella. Ya no será necesario hacer un casting.

El tipo entre los símbolos < > recibe el nombre de tipo parametrizado.

También podemos declarar un tipo parametrizado como argumento de un método:

```
void tomarListadeStrings(List<String> cadenas){
    cadenas.add(new Integer(45));    //¿Compilará?
}
```

Asimismo también podemos hacer que los tipos de retorno sean de tipo seguro:

```
public List<Dog> getDogList( ){
    List<Dog> perros = new ArrayList<Dog>( );
    return perros;
}
```

El compilador nos parara si tratamos de retornar algo no compatible con un List<Dog>. Los que invoquen al método no necesitan hacer un casting.

```
Dog d = getDogList( ).get(0);
```

Genéricos y código heredado

Para actualizar código no genérico a código con genéricos: Añadir un tipo entre < > inmediatamente después del tipo de colección, tanto en la declaración de la variable como en el constructor, incluyendo cualquier sitio en que se declare una variable (esto significa argumentos y tipos de retorno también).

```
public List<String> changeStrings( ArrayList<String> s){ }
```

Mezclando colecciones genéricas y no genéricas

Podemos mezclar código con genéricos correcto con código antiguo sin genéricos:

```
import java.util.*;
public class TestLegacy {
    public static void main(String[ ] args) {
        List<Integer> myList = new ArrayList<Integer>( );
        myList.add(4);
        myList.add(6);
        Adder adder = new Adder( );
        int total = adder.addAll(myList);    // Pasamos la lista a un argumento sin tipo
        System.out.println(total);
    }
}
```

```
import java.util.*;
class Adder {
    int addAll(List list) {          // Método sin genéricos en los argumentos. Asume que
        Iterator it = list.iterator(); // sin garantías que pasaremos Integers. Pero funciona
        int total = 0;
        while (it.hasNext( )) {
            int i = ((Integer)it.next( )).intValue( );
            total += i;
        }
        return total;
    }
}
```

Imaginemos que ahora invocamos al método antiguo pero no leemos un valor, sino que añadimos algo al ArrayList:

```
import java.util.*;
public class TestBadLegacy {
    public static void main(String[ ] args) {
        List<Integer> myList = new ArrayList<Integer>( );
        myList.add(4);
        myList.add(6);
        Inserter in = new Inserter( );
        in.insert(myList);
    }
}
class Inserter{
    void insert(List list) {
        list.add(new Integer(42));
    }
}
```

Compila y corre: Pero ¿qué pasa si modificamos el método insert()?

```
void insert(List list) {
    list.add(new String("42"));
}
```

Desafortunadamente compila y se ejecuta. No se produce excepción en tiempo de ejecución. Hay que recordar que al código antiguo se le permitía poner cualquier cosa en una colección, excepto primitivas.

Sin embargo, en este segundo caso, el compilador **AVISA** tanto si queremos añadir un elemento del tipo correcto, como si es incorrecto. Avisa porque estamos añadiendo **ALGO** a la colección y hay una posibilidad de que nos equivoquemos.

Los avisos del compilador **NO** son considerados fallos.

Además el código anterior correría sin problemas hasta que tratemos de usar los elementos del ArrayList como elementos de tipo Integer!!! Compila y corre sin problemas como si no hubiésemos usado genéricos. La razón es que **LA INFORMACIÓN ENTRE < > NO EXISTE EN TIEMPO DE EJECUCIÓN**. Todo el código genérico es estrictamente para el compilador. El compilador hace todas las verificaciones en el código con genéricos y después hace un “**borrado de tipos**” antes de ejecutar. En tiempo de ejecución, las colecciones son colecciones como en las primeras versiones de Java.

Esto marca una diferencia con los **arrays**, que ofrecen protección tanto en la compilación como en tiempo de ejecución.

```
List test = new ArrayList( );
test.add(43);
int x =(Integer)test.get(0); //El casting es obligatorio sin genéricos, sin embargo...

List<Integer> test = new ArrayList<Integer>( );
test.add(43);
int x =test.get(0); // Aquí el casting no es necesario.
```

NOTA: El método get devuelve siempre una referencia de tipo Object. El unboxing no puede convertir una referencia Object a una primitiva (sólo convierte referencias de clases de envoltura a primitivas).

Polimorfismo y genéricos

Como ya hemos visto el polimorfismo se aplica al tipo base de la colección:

```
List<Integer> miLista = new ArrayList<Integer>( );
```

¿Qué ocurre en esta caso?

```
class Superclase{ }
class Subclase extends Superclase{ }
List<Superclase> miLista = new ArrayList<Subclase>; // NO FUNCIONA
```

El tipo de la declaración de variable **DEBE COINCIDIR** con el tipo del objeto pasado. En otras palabras, lo que hay entre < > debe coincidir a ambos lados de la igualdad.

No funcionan:

```
List<Object> miLista = new ArrayList<JButton>( );
List<Number> miLista = new ArrayList<Integer>( );
```

Si funcionan:

```
List<JButton> miLista = new ArrayList<JButton>( );  
List<Object> miLista = new ArrayList<Object>( );  
List<Integer> miLista = new ArrayList<Integer>( );
```

Métodos genéricos

Supongamos el siguiente método que tiene como argumento un array de la clase Animal. Dicha clase tiene como subclases a Dog, Cat y Bird. Debido al polimorfismo podremos pasar como parámetros cualquier array Animal (**Dog[]**, **Cat[]**, **Bird[]**).

```
public void checkAnimals(Animal[ ] animals){  
    for(Animal a: animals){  
        a.checkup( );  
    }  
}
```

Podremos pasar como argumento cualquier matriz de un tipo que sea subclase de Animal sin problema, ya que **los arrays hacen comprobación de tipos**, pues son de un tipo definido en su declaración.

```
public static void main (String[ ] args){  
    {  
        Dog[ ] dogs = { new Dog( ), new Dog( )};  
        Cat [ ] cats = { new Cat( ), new Cat( ), new Cat( )}; // Ambas subclases de Animal...  
    }  
    .....  
    doc.checkAnimals(dogs);  
    doc.checkAnimals(cats);  
    .....
```

Estamos seguros de que lo que se pasa a la matriz es correcto, y por tanto podemos usar el polimorfismo al invocar al método.

Esto **NO** funciona de la misma manera con las colecciones de tipo seguro. En otras palabras, **un método** que toma digamos un `ArrayList<Animal>` **NO** será capaz de aceptar una colección de cualquier subtipo de Animal. Esto significa que `ArrayList<Dog>` no puede ser pasado en un método con un argumento de tipo `ArrayList<Animal>`. NO se puede pasar un `ArrayList<Dog>` donde se pide un `ArrayList<Animal>` , **se produciría un error, no un aviso**.

NO SE PUEDE asignar un **ArrayList** individual de un subtipo de Animal (<Dog>, <Cat>,...) a un **ArrayList** del supertipo <Animal>, el cual es declarado como tipo de argumento. En otras palabras, **NO SE PUEDE USAR EL POLIMORFISMO**.

Entonces ¿Tendremos que escribir un montón de métodos sobrecargados para cada <tipo> específico? ¿Hay alguna forma de rodear este problema? Veámoslo.

```
Animal [ ] animals = new Animal [3];
animals[0] = new Cat( );
animals[1] = new Dog( );
```

También podemos hacer lo siguiente:

```
List<Animal> animals = new ArrayList<Animal>( );
animals.add(new Cat( ));
animals.add(new Dog( ));
```

Ambos funcionan. Tanto en **arrays** como en colecciones genéricas se puede añadir una instancia de un subtipo en la matriz o la colección declarada con el supertipo.

Ahora nos fijamos en lo que ocurre dentro de un método si usamos arrays:

```
public void addAnimal(Animal[ ] animals){
    animals[0] = new Dog( );    // No hay ningún problema.
}
```

Sin embargo parece que no podemos hacer lo mismo con colecciones con genéricos:

```
public void addAnimal(ArrayList<Animal> animals){
    animals.add(new Dog( ););    // Está permitido en algunas ocasiones....
}
```

Lo cierto es que se puede hacer bajo ciertas condiciones. El código compilará si lo que se pasa al método es también un `ArrayList<Animal>`. En esto es diferente a los arrays, en la versión array se podría pasar un `Dog[ ]` a un método que toma un `Animal[ ]`. Lo único que se puede pasar como a un método con argumento `ArrayList<Animal>` es un `ArrayList<Animal>` (Asumiendo que no estamos tratando de pasar un subtipo de `ArrayList`, ya que el tipo base puede ser polimórfico).

Con los arrays se puede dar una situación indeseable que tratan de prevenir las colecciones con genéricos. El escenario es el siguiente:

```
public void foo( ){
    Dog[ ] dogs = { new Dog( ), new Dog( ) };
    addAnimal(dogs);
}

public void addAnimal(Animal[ ] animals)
{
    animals[0] = new Dog( );
}
```

Esta situación no plantea problemas. Veamos la siguiente:

```
public void foo( ){
    Cat[ ] cats = { new Cat( ), new Cat( )};
    addAnimal(cats); // Sin problema , mandamos Cat[ ] al método
}

public void addAnimal(Animal[ ] animals)
{
    animals[0] = new Dog( ); // Hemos puesto un Dog en una matriz de Cat!!!
}
```

El compilador piensa que está perfectamente bien añadir un Dog a un array Animal[], ya que un Dog puede ser asignado a una referencia Animal. El problema es que si pasamos un array de un subtipo de Animal (Cat, Dog o Bird), el compilador no lo sabe. El compilador no se da cuenta que en alguna parte del heap hay un array de tipo Cat[], no Animal[], y de que le estamos tratando de añadir un Dog. Para el compilador, hemos pasado un array de tipo Animal, así que no tiene forma de reconocer el problema.

La razón por la que el compilador no nos dejará pasar un ArrayList<Dog> a un método que toma como argumento un ArrayList<Animal>, es porque dentro del método, ese parámetro es de tipo ArrayList<Animal>, y eso significa que podríamos poner cualquier clase de Animal en él. No habría forma para el compilador de pararnos de poner un Dog en un ArrayList que originalmente estaba declarado como <Cat>, pero que ahora es referenciado desde el parámetro <Animal>.

La razón de que podamos seguir con la compilación para los arrays es que existe un excepción en tiempo de ejecución (**ArrayStoreException**), que será lanzada al poner el tipo de objeto incorrecto en el array. Si enviamos un array Dog el método que toma Animal[] y añadimos sólo Dog (incluyendo subtipos de Dog) en el array referenciado por Animal, no hay problema. Pero si tratamos de añadir un Cat al objeto que es de hecho un array Dog, se lanzará una excepción.

Pero no hay equivalente para genéricos a causa del borrado de tipos. En otras palabras, en tiempo de ejecución la JVM conoce el tipo de los arrays, pero NO es tipo de una colección. Toda la información acerca de tipos genéricos es eliminada durante la compilación. El siguiente código es legal:

```
public void addAnimal(List<Animal> animals){
    animals.add(new Dog( ));
}

public static void main(String[ ] args){
    List<Animal> animals = new ArrayList<Animal>( );
    animals.add(new Dog( ));
    animals.add(new Dog( ));
    AnimalDoctorGeneric doc = new AnimalDoctorGeneric( );
    doc.addAnimal(animals);    //Compila y se ejecuta sin problemas.
}
```

Pero si tratamos de invocar `addAnimal( )` con un argumento de cualquier otro tipo `ArrayList`, el compilador nos parara, ya que en tiempo de ejecución la JVM no tendría manera de pararnos si metemos un `Dog` a lo que fue creado como una colección `Cat`.

```
public void addAnimal(List<Animal> animals){
    animals.add(new Dog( ));
}

public static void main(String[ ] args){
    List<Dog> animals = new ArrayList<Dog>( );
    animals.add(new Dog( ));
    animals.add(new Dog( ));
    AnimalDoctorGeneric doc = new AnimalDoctorGeneric( );
    doc.addAnimal(animals);    // No compila, el ArrayList es de tipo <Dog>.
}
```

¿Cómo podemos rodear este problema? Con el mecanismo **wildcard**.

- El mecanismo **wildcard** (comodín)

El problema es lo que podemos hacer con el método **`add( )`**. Necesitamos una manera de decirle al compilador: **“Estoy usando la colección pasada como argumento sólo para invocar métodos en los elementos, y PROMETO no añadir nada a la colección”**

Hay un mecanismo que nos permite usar el polimorfismo en métodos con argumentos genéricos a cambio de comprometernos a no añadir ningún elemento a la colección. Dicho mecanismo es el wildcard **`<?>`**. La signatura de un método cambia de:

```
public void addAnimal(List<Animal> animals)
```

a:

```
public void addAnimal(List<? extends Animal> animals)
```

Al poner **`<? extends Animal>`** estamos diciendo: “se me puede asignar una colección que sea un subtipo de `List` con tipo `<Animal>` o **cualquier clase que extienda a `Animal`**, y prometo no añadir nada a la colección”.

```
public void addAnimal(List< ? extends Animal> animals){
    animals.add(new Dog( )); // NO!!! No podemos añadir nada
                             // si usamos <? extends Animal>
}
```

Si compilamos el código anterior obtenemos un error. Tenemos que tener en cuenta algunas cosas más:

En primer lugar, **<? extends Animal>** significa que podemos pasar cualquier subtipo de Animal, sin embargo, ese subtipo puede ser una subclase de una clase (abstract o concreta) o un tipo que implementa el interface que sigue a la palabra extends. En otras palabras, la palabra clave **extends** en el contexto wildcard representa tanto a implementaciones de subclases como de interfaces (**no existe la sintaxis <? implements Serializable>**). Si queremos declarar un método que toma algo que es de un tipo que implementa Serializable, tendremos que usar **extends**:

```
void foo(List<? extends Serializable> list)
```

Por otra parte, hay otro escenario en la cual, usando un **wildcard** podemos añadir a la colección, pero de una forma completamente segura, usando la palabra clave **super**.

```
public void addAnimal(List<? super Dog> animals){
    animals.add(new Dog( ));
}

public static void main(String[ ] args){
    List<Animal> animals = new ArrayList<Animal>( );
    animals.add(new Dog( ));
    animals.add(new Dog( ));
    AnimalDoctorGeneric doc = new AnimalDoctorGeneric( );
    doc.addAnimal(animals);    //Compila y se ejecuta sin problemas.
}
```

Con **<? super Dog>** decimos al compilador que **acepte cualquier List de tipo genérico Dog o cualquier supertipo de Dog (NO se aceptará nada más bajo en el árbol de herencia)**.

El wildcard permite hacer asignaciones polimórficas pero con ciertas restricciones que no tenemos para los arrays. ¿Son los dos casos siguientes idénticos?

```
public void foo(List<?> list){ }
```

? sin extends o super simplemente significa “cualquier tipo”. Es decir, cualquier tipo de lista puede ser usada como argumento. Además, **NO** se permite añadir (add()) nada a la lista.

```
public void foo(List<Object> list){ }
```

Significa que el método solo puede tomar un List<Object>. En este caso si podemos añadir elementos a List.

Sin embargo **List<? extends Object>** y **List<?>** son absolutamente idénticos. Ambos dicen “Me puedo referir a cualquier tipo de objeto”.

Algunos ejemplos:

```
List<?> list = new ArrayList<Dog>( );  
List<? extends Animal> list = new ArrayList<Dog>( );  
List<?> list = new ArrayList<? extends Animal>( );  
List<? extends Dog> list = new ArrayList<Integer>( );  
List<? super Dog> list = new ArrayList<Animal>( );  
List<? super Animal> list = new ArrayList<Dog>( );
```

¿Compilan?